



Android Dev Tips

Memory Leaks



What is Memory Leak?

In Android, objects are stored in the phone's RAM, and when you're done using them (like when an activity closes), Android's garbage collector cleans them up to free memory.

A memory leak happens when the system can't clean up objects that are no longer needed, often because your code still holds a reference to them.

Example:

If a background task keeps a reference to a closed activity, the system can't free that memory — that's a leak.

Fix:

Cancel tasks, remove listeners, and avoid static variables pointing to activities or views. This helps Android manage memory properly.



If your app is leaking memory

1. **Slow Performance:** The app becomes slower as more memory is used up, affecting its responsiveness.
2. **Crashes:** Over time, memory leaks can lead to `OutOfMemoryError`, causing the app to crash.
3. **Increased Battery Usage:** Since the app uses more memory than necessary, it can drain the battery faster.
4. **App Shutdowns:** The system might force the app to close if it consumes too much memory, resulting in a poor user experience.



Detecting Memory Leaks

Let's intentionally make a leak and understand how to detect the leaks. We will be using LeakCanary here.

```
dependencies {  
    // debugImplementation because LeakCanary should only run in debug builds.  
    debugImplementation("com.squareup.leakcanary:leakcanary-android:2.14")  
}
```



Creating a Leak

```
class MemoryLeakActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        leakedActivityReference = this
        Log.d(TAG, "Activity Reference Leaked")
        setContent {
            MemoryLeakTheme {
                Scaffold(modifier = Modifier.fillMaxSize()) {...}
            }
        }
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "Activity Destroyed, but reference is leaked")
    }

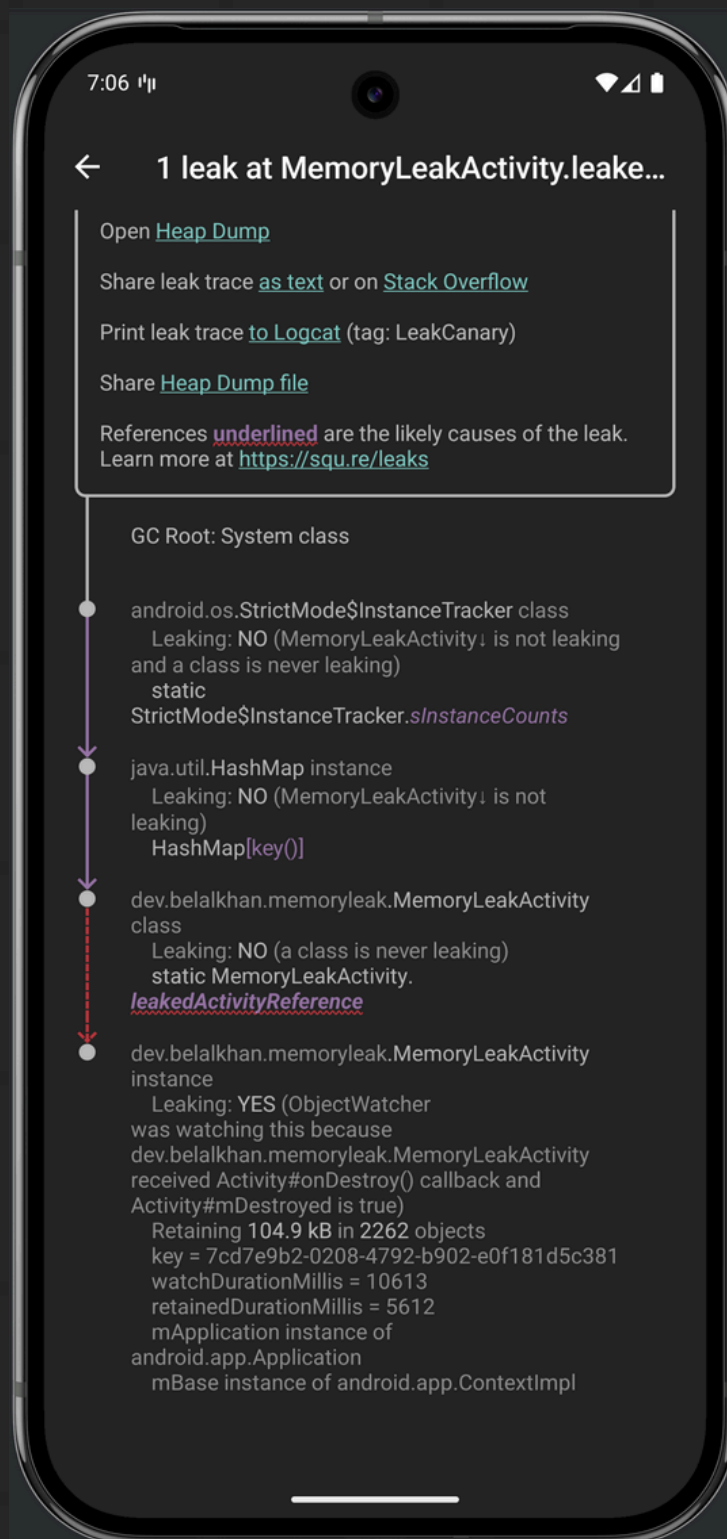
    companion object {
        private val TAG = this::class.simpleName
        var leakedActivityReference: Activity? = null
        fun launch(context: Context) {...}
    }
}
```

The MemoryLeakActivity holds a static reference (leakedActivityReference) to itself, preventing the activity from being garbage collected after it finishes, causing a memory leak.



Detecting Memory Leak

Launch the above created activity and close.





👉 Share with your fellow devs
and let's grow the Android community! 🌟

for the latest tips, tricks, and tutorials!
Subscribe To

 /@SimplifiedCoding

THANK
YOU

 /@SimplifiedCoding